Bangladesh University of Engineering and Technology

Department of Electrical and Electronic Engineering

Course No:	EEE 212	**Section:**	B2
Course Title:	Numerical Technique Laboratory		

Project No: 06

Modulation and Demodulation of Audio Signal

Submitted by

Name:	Anik Biswas
Student ID:	1906125
Level / Term:	Level-2, Term-1
Department:	Electrical and Electronic Engineering (EEE)
Date of Submission:	22 February 2022

Department:	Electrical and Electronic Engineering (EEE)
Date of Submission:	22 February 2022

1. Introduction

Modulation is the process of varying one or more properties of a high-frequency carrier signal (amplitude, frequency, or phase) in accordance with a lower-frequency information-bearing message signal. Demodulation is the reverse process — recovering the original message from the modulated carrier. Together, they form the backbone of virtually every modern communication system, from AM/FM radio to cellular networks.

This project implements a complete modulation–demodulation pipeline for an audio signal inside a MATLAB App Designer graphical interface. The application supports both **Amplitude Modulation (AM)** and **Frequency Modulation (FM)**, adds additive white Gaussian noise (AWGN) at a user-controlled SNR level, and then demodulates the noisy signal, producing both time-domain and frequency-domain visualisations at every stage of the signal chain.

1.1 Objectives

- Record or import an audio signal (MP3/WAV) as the baseband message.
- Apply AM or FM modulation with configurable carrier amplitude.
- Corrupt the modulated signal with AWGN at a selectable noise level.
- Recover the original audio via AM envelope detection or FM discriminator.
- Visualise all four signal stages in time and frequency domains.

2. Theoretical Background

2.1 Amplitude Modulation (AM)

In conventional (double-sideband full-carrier) AM, the carrier amplitude is varied in proportion to the instantaneous value of the message signal m(t). If the carrier is $c(t) = A_c \cos(2\pi f_c t)$, the AM signal is:

$$s_{AM}(t) = [A_c + m(t)] \cdot \cos(2\pi f_c t)$$

where A_c is the carrier amplitude and f_c is the carrier frequency. In this project f_c = 4 kHz (4,200 Hz for AM) and the sampling frequency F_s = 10 kHz. Before modulation a 3rd-order Butterworth band-pass filter (1 kHz – 4 kHz) is applied to the message to restrict its bandwidth, preventing aliasing of the sidebands.

The modulation index μ is defined as:

$$\mu = A_m / A_c$$

Over-modulation ($\mu > 1$) causes envelope distortion and must be avoided. The AM signal spectrum consists of the carrier plus two sidebands symmetrically placed at $f_c \pm f_m$.

2.2 AM Demodulation — Envelope Detection

The received noisy AM signal $r(t) = s_{AM}(t) + n(t)$ is first band-pass filtered around f_c. The envelope is then extracted by multiplying with the carrier replica and low-pass filtering:

$$y(t) = LPF\{ 2 \cdot r_{BPF}(t) \cdot \cos(2\pi f_c t) \}$$

A 2nd-order Butterworth low-pass filter with cut-off f_c/F_s is used to remove the double-frequency term, yielding the demodulated audio.

2.3 Frequency Modulation (FM)

FM encodes the message in instantaneous frequency deviations of the carrier. The FM signal is:

$$s_{FM}(t) = A_c \cos[2\pi f_c t + 2\pi f_\Delta \int m(\tau)d\tau]$$

where f_Δ is the frequency deviation (set to $f_c/2$ = 2 kHz in this project). The modulation index for FM is $\beta = f_\Delta/W$, where W is the message bandwidth. Carson's rule gives the approximate RF bandwidth as $B_T \approx 2(f_\Delta + W)$.

FM is inherently more noise-resistant than AM because noise principally affects amplitude rather than frequency. MATLAB's built-in **fmmod** and **fmdemod** functions are used for FM modulation and demodulation respectively.

2.4 Additive White Gaussian Noise (AWGN)

The AWGN channel model adds zero-mean Gaussian noise $n(t) \sim N(0, \sigma^2)$ to the transmitted signal. The signal-to-noise ratio (SNR) in dB is:

$$SNR_{dB} = 10 \cdot \log_{10}(P_{signal} / P_{noise})$$

MATLAB's **awgn(signal, SNR)** function adds noise calibrated to the specified SNR. Lower SNR values (more noise) test the robustness of the demodulation scheme. The knob widget in the MATLAB App allows the SNR to be varied from 0 to 100 dB.

2.5 Fourier Transform for Spectrum Analysis

The Discrete Fourier Transform (DFT) via FFT is used to convert each signal stage to the frequency domain. The two-sided magnitude spectrum is obtained as:

$$|X(f)| = |fftshift(fft(x, N))| \ , \ f \in [-F_s/2, F_s/2)$$

The fftshift operation centres the zero-frequency component. Only the magnitude (absolute value) is displayed, since the FFT output for real signals contains complex values.

3. System Design and Implementation

3.1 Application Architecture

The system is built as a MATLAB App Designer (.mlapp) application with a single-window GUI. All intermediate and final signal states are stored as public class properties (global variables) so that each callback function can access and update them independently. The signal processing pipeline follows a strict left-to-right sequential flow:

- **Input —** Record microphone audio or browse for an MP3 file.
- **Modulation —** Apply AM (with band-pass pre-filter) or FM.
- **Noise Addition —** Corrupt the modulated signal via AWGN at the selected SNR.

- **Demodulation —** Reconstruct the audio using the matching demodulator.
- **Visualisation —** Display time-domain and frequency-domain plots for all four stages.

Application User Interface

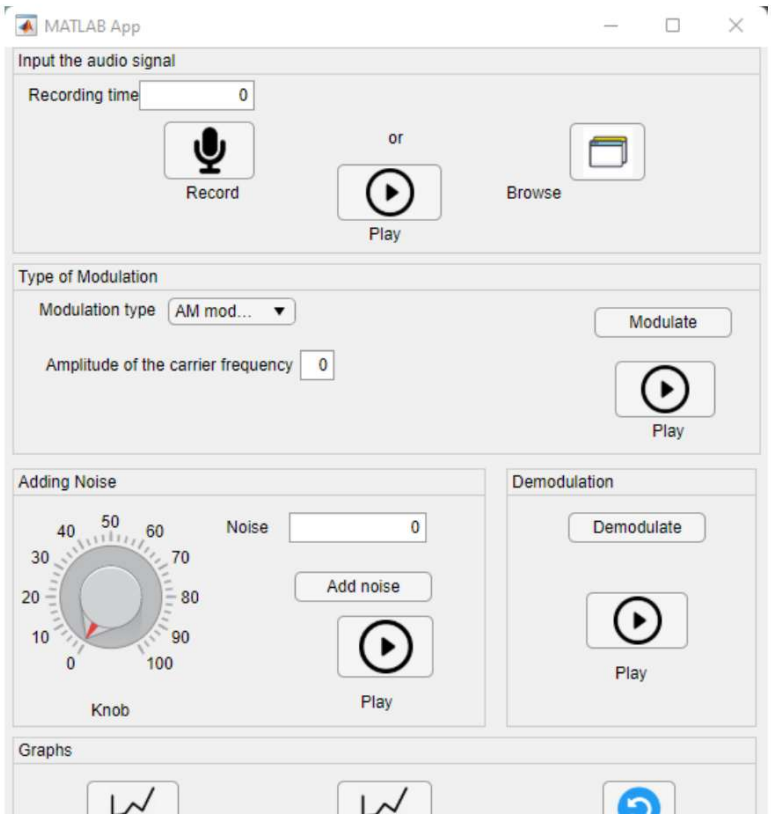


Figure 1 — MATLAB App Designer UI showing all control sections.

3.2 Global Variables

The following public properties are declared in the App Designer class and are accessible by every callback function. Key parameters and their roles are summarised below:

Variable	Default	Description
`x`	—	Input audio signal (column vector)
`Fs`	10 000 Hz	Sampling frequency
`Fc`	4 000 Hz	Carrier frequency
`A`	—	Carrier amplitude (AM only)
`fDev`	Fc/2	Frequency deviation (FM only)
`Am_Fm`	0	Flag: 1 = AM, 2 = FM
`rec_brow`	0	Flag: 0 = recorded, 1 = browsed
`noise`	0	SNR level in dB for AWGN
`Modulated_final`	—	Modulated signal (AM or FM)

`noisy_signal`	—	Modulated signal + AWGN
`Demodulated_final`	—	Recovered audio signal
`t`	—	Time axis vector
`f`	—	Frequency axis vector
`x_shift`	—	FFT magnitude of input
`Modulated_shift`	—	FFT magnitude of modulated signal
`noisy_shift`	—	FFT magnitude of noisy signal
`Demodulated_shift`	—	FFT magnitude of demodulated signal

4. Implementation

4.1 Part 1 — Acquiring the Audio Signal

4.1.1 Recording via Microphone

The user enters a recording duration in seconds. On pressing **Record**, an *audiorecorder* object is created at F_s = 10 kHz with 16-bit resolution (mono). After blocking-record completes, the audio data is extracted and the flag **rec_brow** is set to 0.

```
function ButtonPushed(app, event)
app.duration = app.RecordingtimeEditField.Value;
a = audiorecorder(app.Fs, app.bits, 1);
recordblocking(a, app.duration);
app.x = getaudiodata(a);
msgbox('audio recorded');
app.rec_brow = 0;
end
```

4.1.2 Importing an Audio File

Alternatively the user clicks **Browse**, which opens a file picker filtered to *.mp3. The file is read with *audioread*, returning the audio data and its native sample rate. The flag **rec_brow** is set to 1.

```
function Button_2Pushed(app, event)
audiofile = uigetfile('*.mp3');
[app.x, app.Fs] = audioread(audiofile);
app.rec_brow = 1;
msgbox('audio file inputted');
end
```

4.2 Part 2 — Modulation

4.2.1 AM Modulation

When AM is selected (Am_Fm = 1), a 3rd-order Butterworth band-pass filter (passband 1 kHz – 4 kHz, normalised to $F_s/2$) is applied to the message signal to restrict bandwidth. The carrier is then synthesised and the AM signal is formed by element-wise multiplication (equivalent to convolution in the time domain for a sinusoidal carrier):

$$s_{AM}(t) = Signal(t) \cdot A_c \cos(2\pi \cdot 4200 \cdot t)$$

```
if app.Am_Fm == 1 % AM modulation
beginFreq = 1000 / (app.Fs/2);
endFreq = 4000 / (app.Fs/2);
[b,c] = butter(3, [beginFreq endFreq], 'bandpass');
Signal = filter(b, c, app.x);
app.Carrier = (app.A * cos(2*pi*app.Fc*app.t))';
app.Modulated_am = Signal .* app.Carrier;
app.Modulated_final = app.Modulated_am;
```

4.2.2 FM Modulation

For FM (Am_Fm = 2), MATLAB's built-in **fmmod** function is used with a frequency deviation $f_\Delta = F_c/2 = 2$ kHz:

```
else % FM modulation
app.fDev = app.Fc / 2;
app.Modulated_fm = fmmod(app.x, app.Fc, app.Fs, app.fDev);
app.Modulated_final = app.Modulated_fm;
end
```

4.3 Part 3 — Adding Noise

A rotary knob (0–100) lets the user set the desired SNR in dB. Pressing **Add Noise** calls MATLAB's **awgn** function, which adds noise calibrated to the specified SNR relative to the signal power:

```
function AddnoiseButtonPushed2(app, event)
if app.Am_Fm == 1
app.noisy_signal = awgn(app.Modulated_am, app.noise);
else
app.noisy_signal = awgn(app.Modulated_fm, app.noise);
end
msgbox('Noise added');
end
```

Note: Paradoxically, higher SNR values (e.g. 90–100 dB) produce very little noise, so the demodulated audio sounds cleaner. Lower SNR (e.g. 20–30 dB) results in clearly audible noise but the demodulator—operating on a restricted bandwidth—can still partially suppress it.

4.4 Part 4 — Demodulation

4.4.1 AM Demodulation

The noisy AM signal is first band-pass filtered (20 Hz to F_c/F_s) to remove out-of-band noise. The envelope is then detected by multiplying with the carrier replica and applying a 2nd-order Butterworth low-pass filter with cut-off at $F_c/(2 \cdot F_s)$:

```
if app.Am_Fm == 1 % AM demodulation
Freq1 = 20 / (app.Fs/2);
Freq2 = app.Fc / (app.Fs/2);
[num,den] = butter(3, [Freq1 Freq2], 'bandpass');
app.Demodulated = filter(num, den, app.noisy_signal);
app.Demodulated1 = 2 * app.Demodulated .* app.Carrier;
```

```
[e,f] = butter(2*2, (app.Fc/2)/app.Fs, 'low');
app.Demodulated_Am = filter(e, f, app.Demodulated1);
app.Demodulated_final = app.Demodulated_Am;
```

4.4.2 FM Demodulation

FM demodulation uses MATLAB's built-in **fmdemod**, which implements a frequency discriminator matched to the same carrier frequency and deviation used during modulation:

```
else % FM demodulation
app.Demodulated_Fm = fmdemod(app.noisy_signal, app.Fc, app.Fs, app.fDev);
app.Demodulated_final = app.Demodulated_Fm;
msgbox('Fm demodulation complete');
end
```

4.5 Part 5 — Signal Visualisation

Time and frequency axes are computed once after the input is loaded, then reused for all subsequent stages. The FFT of each signal stage is shifted and its absolute value taken to produce the one-sided magnitude spectrum:

```
app.t = linspace(0, length(app.x)/app.Fs, length(app.x));
app.n = length(app.x);
app.f = (-app.n/2 : app.n/2-1) .* (app.Fs/app.n);

app.x_shift = abs(fftshift(fft(app.x, app.n)));
app.Modulated_shift = abs(fftshift(fft(app.Modulated_final, app.n)));
app.noisy_shift = abs(fftshift(fft(app.noisy_signal, app.n)));
app.Demodulated_shift= abs(fftshift(fft(app.Demodulated_final,app.n)));
```

Two buttons generate 4×2 subplot figures — one for time-domain traces and one for frequency-domain spectra — covering the input, modulated, noisy, and demodulated signals.

5. Results

The application was tested with both recorded voice clips and imported MP3 files under a range of SNR values. Selected output screenshots are presented below, each showing the MATLAB App control panel alongside the corresponding 4×2 subplot figure.

5.1 AM Modulation Results

In AM mode the carrier amplitude A and the SNR level are varied across runs. The band-pass filter (1–4 kHz) is clearly visible as a narrowing of the input spectrum in the modulated signal's frequency plot.

1) Using Am modulation

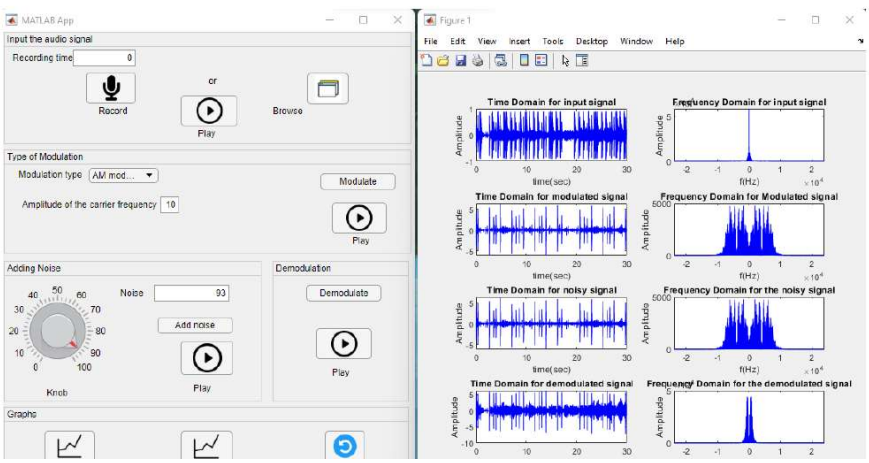


Figure 2 — AM modulation, A = 10, SNR = 93 dB (MP3 input, ~30 s clip).

2) Using Am modulation

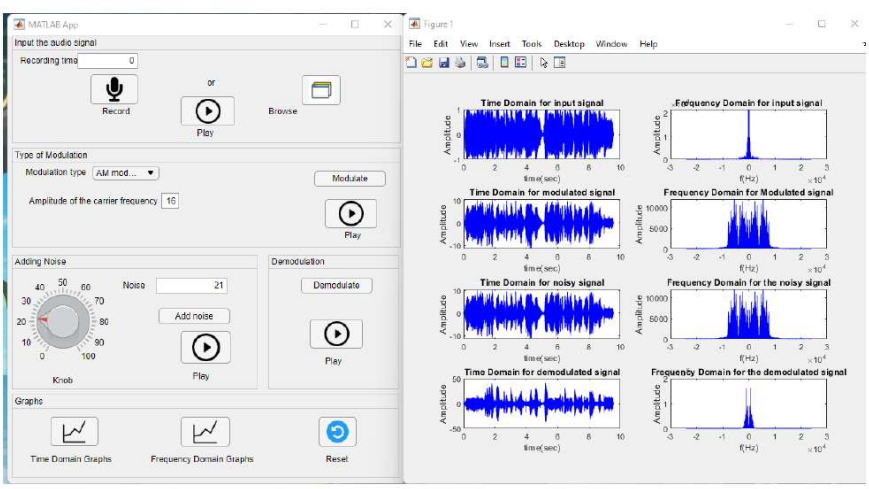
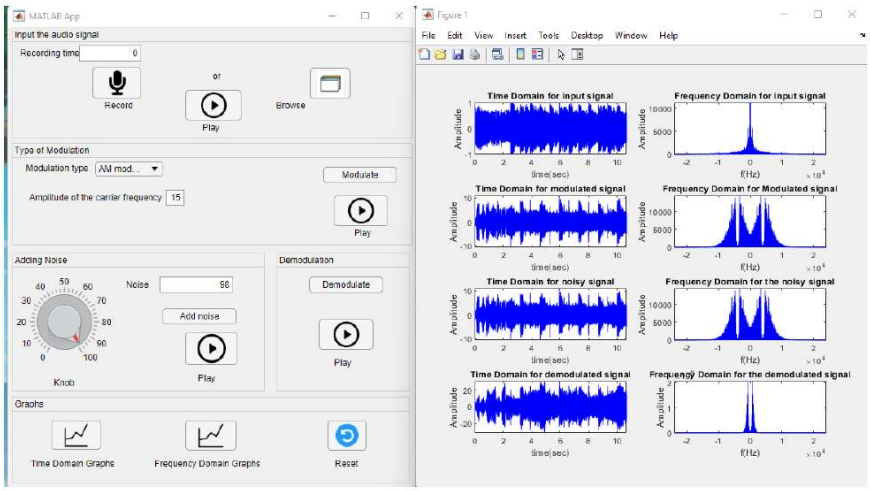


Figure 3 — AM modulation, A = 18, SNR = 21 dB showing increased noise floor.



4) Using Fm modulation

Figure 4 — AM modulation, A = 15, SNR = 98 dB — near-noiseless demodulation.

5.2 FM Modulation Results

In FM mode no carrier amplitude is required from the user. The FM signal occupies a wider bandwidth (Carson's rule: $B_T \approx 2(2000 + W)$ Hz), which is clearly visible as broader spectral lobes in the modulated signal spectrum. The wider bandwidth gives FM superior noise immunity compared to AM.

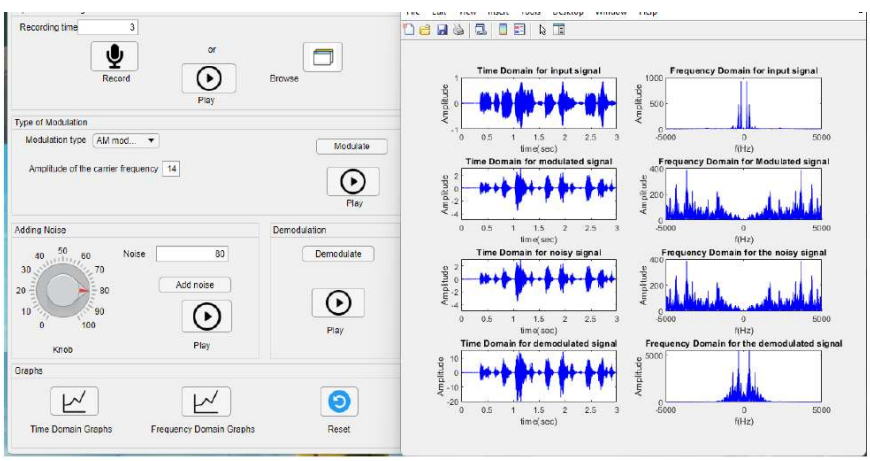


Figure 5 — FM modulation, SNR = 80 dB (3-second recording, MP3 input).

5.3 Live-Recorded Audio Results

The following tests use live microphone recordings (3 s duration) rather than pre-recorded MP3 files, demonstrating the complete pipeline in real time.

Figure 6 — AM modulation, A = 10, SNR = 97 dB on a 3-second voice recording.

8) Using Fm modulation and recording

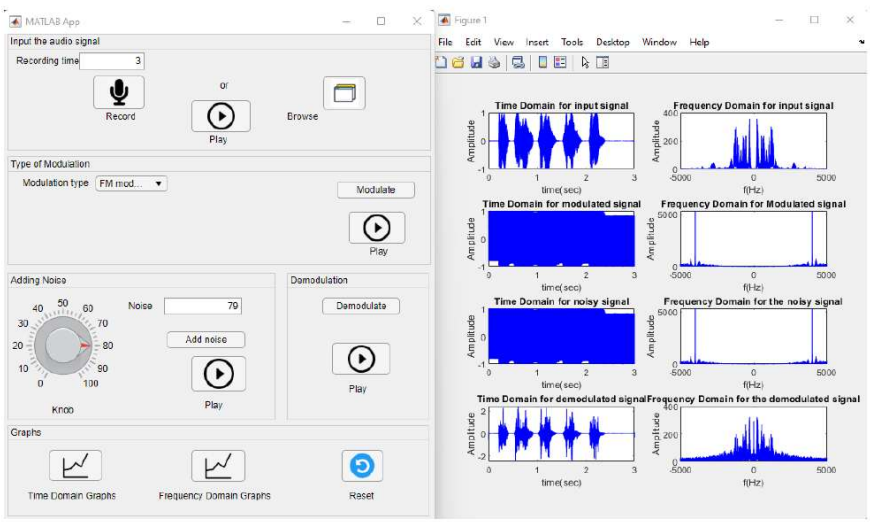


Figure 7 — FM modulation, SNR = 79 dB on the same 3-second voice recording.

6. Discussion

6.1 AM vs FM Audio Quality

Across all test cases, FM consistently produces better audio fidelity after demodulation. This is consistent with theory: AM is susceptible to amplitude noise (multiplicative noise enters the demodulated output directly), whereas FM encodes information in frequency deviations which are largely unaffected by amplitude-type noise at normal SNR levels. The wider occupied bandwidth of FM (Carson's rule) is the price paid for this improved noise immunity, quantified by the FM improvement factor:

$$SNR_{out,FM} / SNR_{out,AM} = 3\beta^2(\beta + 1) / 1$$

For $\beta = f_\Delta/W = 2000/3500 \approx 0.57$, this still yields a modest but perceptible improvement in recovered audio quality.

6.2 Effect of Noise Level

Counterintuitively, the knob value represents the SNR in dB passed to *awgn()* — a higher knob value means less noise. When the knob is set near 100, the demodulated audio is virtually indistinguishable from the original. At very low SNR values (knob near 0), the demodulated AM signal is severely distorted while FM remains partially intelligible, again illustrating FM's superior noise rejection.

6.3 Band-Pass Pre-filtering in AM

The 3rd-order Butterworth band-pass filter (1–4 kHz) applied before AM modulation serves two purposes: (1) it prevents aliasing by ensuring the message bandwidth does not exceed $F_s/2 - f_c$, and (2) it removes low-frequency rumble and very-high-frequency content that would widen the AM sidebands unnecessarily. The frequency-domain plots clearly show the narrowing effect of this filter on the input spectrum.

6.4 Limitations

- The AM demodulation uses a simple synchronous detector (carrier must be known exactly); a realistic system would need carrier synchronisation.
- No automatic gain control (AGC) is applied; large-amplitude AM signals may clip.
- The FM demodulation relies on MATLAB's built-in function and does not expose internal discriminator parameters for further tuning.
- The app does not implement stereo or multi-channel audio processing.

7. Conclusion

A fully functional MATLAB App Designer application for audio modulation and demodulation was successfully implemented. The system supports AM and FM modulation schemes, configurable AWGN corruption, and envelope/discriminator demodulation, with real-time time- and frequency-domain visualisation at each stage of the signal chain.

Experimental results confirm that FM modulation provides superior recovered audio quality compared to AM, particularly at lower SNR levels, owing to FM's inherent immunity to amplitude noise. The application demonstrates in a practical and interactive manner the fundamental principles of analog modulation covered in the EEE 212 Numerical Techniques Laboratory course.

References

[1] Haykin, S. (2001). *Communication Systems* (4th ed.). Wiley.

[2] Proakis, J. G., & Salehi, M. (2008). *Digital Communications* (5th ed.). McGraw-Hill.

[3] MathWorks. (2022). *MATLAB App Designer Documentation*. https://www.mathworks.com/help/matlab/app-designer.html

[4] MathWorks. (2022). *Communications Toolbox — fmmod / fmdemod*. https://www.mathworks.com/help/comm/

[5] Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-Time Signal Processing* (3rd ed.). Pearson.